



UNIVERSIDADE FEDERAL DO CEARÁ

CAIO RIAN
FELIPE OLIVEIRA

QUALIDADE DE SOFTWARE

QUIXADÁ
2025

Long Parameter List (StudyPlan)

Fix the "ExcessiveParameterList" code smell using the "Introduce Parameter Object" refactoring technique.

não crie novos só arquivos, apenas adapte esse existente

🔗 Para ajustar o código e remover o "ExcessiveParameterList" utilizando a técnica de refatoração "Introduce Parameter Object", vamos criar uma classe interna `StepDetails` dentro da classe `StudyPlan` e encapsular os parâmetros nela. Vamos ajustar o método `handleSetSteps` para utilizar esse novo objeto.

```
package org.example.studyregistry;
```

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
```

```
public class StudyPlan extends Registry {
    private StudyObjective objective;
    private List<String> steps;
```

```
    public StudyPlan(String planName, StudyObjective objective, List<StudyMaterial>
materials) {
        this.name = planName;
        this.objective = objective;
        this.steps = new ArrayList<>();
    }
```

```
    @Override
    public String toString() {
        return "Plan: " + name + ",\nObjective: " + objective.getDescription() + ",\nSteps:
" + String.join(", ", steps);
    }
```

```
    public List<String> getSteps() {
        return steps;
```

```

    }

    public StudyObjective getObjective() {
        return objective;
    }

    public void assignObjective(StudyObjective objective) {
        this.objective = objective;
    }

    public void addSingleStep(String toAdd) {
        steps.add(toAdd);
    }

    public void assignSteps(StepDetails stepDetails) {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy
HH:mm:ss");

        this.steps = new ArrayList<>(Arrays.asList(
            stepDetails.getFirstStep(),
            stepDetails.getResetStudyMechanism(),
            stepDetails.getConsistentStep(),
            stepDetails.getSeasonalSteps(),
            stepDetails.getBasicSteps(),
            "Number of steps: " + stepDetails.getNumberOfSteps().toString(),
            "Is it important to you? " + stepDetails.isImportant(),
            stepDetails.getStartDate().format(formatter),
            stepDetails.getEndDate().format(formatter),
            stepDetails.getMainObjectiveTitle(),
            stepDetails.getMainGoalTitle(),
            stepDetails.getMainMaterialTopic(),
            stepDetails.getMainTask()
        ));
    }

```

```

    public void handleAssignSteps(List<String> stringProperties, Integer
numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime
endDate) {
        StepDetails stepDetails = new StepDetails(
            stringProperties.get(0),
            stringProperties.get(1),
            stringProperties.get(2),
            stringProperties.get(3),
            stringProperties.get(4),
            stringProperties.get(5),
            stringProperties.get(6),
            stringProperties.get(7),
            stringProperties.get(8),
            numberOfSteps,
            isImportant,
            startDate,
            endDate
        );
        assignSteps(stepDetails);
    }

```

```

private class StepDetails {
    private String firstStep;
    private String resetStudyMechanism;
    private String consistentStep;
    private String seasonalSteps;
    private String basicSteps;
    private String mainObjectiveTitle;
    private String mainGoalTitle;
    private String mainMaterialTopic;
    private String mainTask;
    private Integer numberOfSteps;
    private boolean important;
    private LocalDateTime startDate;
    private LocalDateTime endDate;
}

```

```

    public StepDetails(String firstStep, String resetStudyMechanism, String
consistentStep, String seasonalSteps, String basicSteps, String mainObjectiveTitle,
String mainGoalTitle, String mainMaterialTopic, String mainTask, Integer
numberOfSteps, boolean important, LocalDateTime startDate, LocalDateTime
endDate) {
        this.firstStep = firstStep;
        this.resetStudyMechanism = resetStudyMechanism;
        this.consistentStep = consistentStep;
        this.seasonalSteps = seasonalSteps;
        this.basicSteps = basicSteps;
        this.mainObjectiveTitle = mainObjectiveTitle;
        this.mainGoalTitle = mainGoalTitle;
        this.mainMaterialTopic = mainMaterialTopic;
        this.mainTask = mainTask;
        this.numberOfSteps = numberOfSteps;
        this.important = important;
        this.startDate = startDate;
        this.endDate = endDate;
    }

    public String getFirstStep() {
        return firstStep;
    }

    public String getResetStudyMechanism() {
        return resetStudyMechanism;
    }

    public String getConsistentStep() {
        return consistentStep;
    }

    public String getSeasonalSteps() {
        return seasonalSteps;
    }

```

```
public String getBasicSteps() {  
    return basicSteps;  
}  
  
public String getMainObjectiveTitle() {  
    return mainObjectiveTitle;  
}  
  
public String getMainGoalTitle() {  
    return mainGoalTitle;  
}  
  
public String getMainMaterialTopic() {  
    return mainMaterialTopic;  
}  
  
public String getMainTask() {  
    return mainTask;  
}  
  
public Integer getNumberOfSteps() {  
    return numberOfSteps;  
}  
  
public boolean isImportant() {  
    return important;  
}  
  
public LocalDateTime getStartDate() {  
    return startDate;  
}  
  
public LocalDateTime getEndDate() {  
    return endDate;  
}  
}
```

}

```
vc: gerou novos code smells, isso não pode acontecer, ajuste

{
  "formatVersion": 0,
  "pmdVersion": "7.9.0",
  "timestamp": "2025-01-09T09:32:45.108-03:00",
  "files": [
    {
      "filename":
        ".\\src\\main\\java\\org\\example\\studyregistry\\StudyPlan.java",
      "violations": [
        {
          "beginline": 79,
          "begincolumn": 19,
          "endline": 79,
          "endcolumn": 24,
          "description": "The class 'u0027StepDetails'u0027 is suspected
            to be a Data Class (WOC\u003d0.000%, NOPA\u003d0,
            NOAM\u003d13, WMC\u003d14)",
          "rule": "DataClass",
          "ruleset": "Design",
          "priority": 3,
          "externalInfoUrl": "https://docs.pmd-code.org/pmd-doc-
            7.9.0/pmd_rules_java_design.html#dataclass"
        },
        {
          "beginline": 94,
          "begincolumn": 16,
          "endline": 94,
          "endcolumn": 27,
          "description": "The constructor 'u0027StepDetails(String,
            String, String, String, String, String, String, Integer,
            boolean, LocalDateTime, LocalDateTime)'u0027 has a NCSS line
            count of 14.",
          "rule": "NcssCount",
          "ruleset": "Design",
          "priority": 3,
          "externalInfoUrl": "https://docs.pmd-code.org/pmd-doc-
```

Para resolver os problemas identificados no relatório PMD, precisaremos ajustar a classe StepDetails para que não seja considerada uma "Data Class" e reduzir a contagem de linhas no construtor, além de evitar listas de parâmetros longas. Vamos modificar a classe StepDetails e a classe StudyPlan para resolver esses problemas.

```
package org.example.studyregistry;
```

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
```

```
public class StudyPlan extends Registry {
    private StudyObjective objective;
```

```
private List<String> steps;
```

```
public StudyPlan(String planName, StudyObjective objective, List<StudyMaterial>
materials) {
    this.name = planName;
    this.objective = objective;
    this.steps = new ArrayList<>();
}
```

```
@Override
```

```
public String toString() {
    return "Plan: " + name + ",\nObjective: " + objective.getDescription() + ",\nSteps:
" + String.join(", ", steps);
}
```

```
public List<String> getSteps() {
    return steps;
}
```

```
public StudyObjective getObjective() {
    return objective;
}
```

```
public void assignObjective(StudyObjective objective) {
    this.objective = objective;
}
```

```
public void addSingleStep(String toAdd) {
    steps.add(toAdd);
}
```

```
public void assignSteps(StepDetails stepDetails) {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy
HH:mm:ss");

    this.steps = new ArrayList<>(Arrays.asList(
```



```

        stepDetails.getFirstStep(),
        stepDetails.getResetStudyMechanism(),
        stepDetails.getConsistentStep(),
        stepDetails.getSeasonalSteps(),
        stepDetails.getBasicSteps(),
        "Number of steps: " + stepDetails.getNumberOfSteps().toString(),
        "Is it important to you? " + stepDetails.isImportant(),
        stepDetails.getStartDate().format(formatter),
        stepDetails.getEndDate().format(formatter),
        stepDetails.getMainObjectiveTitle(),
        stepDetails.getMainGoalTitle(),
        stepDetails.getMainMaterialTopic(),
        stepDetails.getMainTask()
    ));
}

```

```

public void handleAssignSteps(List<String> stringProperties, Integer
numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime
endDate) {
    StepDetails stepDetails = new StepDetails()
        .setFirstStep(stringProperties.get(0))
        .setResetStudyMechanism(stringProperties.get(1))
        .setConsistentStep(stringProperties.get(2))
        .setSeasonalSteps(stringProperties.get(3))
        .setBasicSteps(stringProperties.get(4))
        .setMainObjectiveTitle(stringProperties.get(5))
        .setMainGoalTitle(stringProperties.get(6))
        .setMainMaterialTopic(stringProperties.get(7))
        .setMainTask(stringProperties.get(8))
        .setNumberOfSteps(numberOfSteps)
        .setImportant(isImportant)
        .setStartDate(startDate)
        .setEndDate(endDate);
    assignSteps(stepDetails);
}

```

```
public static class StepDetails {  
    private String firstStep;  
    private String resetStudyMechanism;  
    private String consistentStep;  
    private String seasonalSteps;  
    private String basicSteps;  
    private String mainObjectiveTitle;  
    private String mainGoalTitle;  
    private String mainMaterialTopic;  
    private String mainTask;  
    private Integer numberOfSteps;  
    private boolean important;  
    private LocalDateTime startDate;  
    private LocalDateTime endDate;  
  
    public StepDetails setFirstStep(String firstStep) {  
        this.firstStep = firstStep;  
        return this;  
    }  
  
    public StepDetails setResetStudyMechanism(String resetStudyMechanism) {  
        this.resetStudyMechanism = resetStudyMechanism;  
        return this;  
    }  
  
    public StepDetails setConsistentStep(String consistentStep) {  
        this.consistentStep = consistentStep;  
        return this;  
    }  
  
    public StepDetails setSeasonalSteps(String seasonalSteps) {  
        this.seasonalSteps = seasonalSteps;  
        return this;  
    }  
  
    public StepDetails setBasicSteps(String basicSteps) {
```

```
    this.basicSteps = basicSteps;
    return this;
}
```

```
public StepDetails setMainObjectiveTitle(String mainObjectiveTitle) {
    this.mainObjectiveTitle = mainObjectiveTitle;
    return this;
}
```

```
public StepDetails setMainGoalTitle(String mainGoalTitle) {
    this.mainGoalTitle = mainGoalTitle;
    return this;
}
```

```
public StepDetails setMainMaterialTopic(String mainMaterialTopic) {
    this.mainMaterialTopic = mainMaterialTopic;
    return this;
}
```

```
public StepDetails setMainTask(String mainTask) {
    this.mainTask = mainTask;
    return this;
}
```

```
public StepDetails setNumberOfSteps(Integer numberOfSteps) {
    this.numberOfSteps = numberOfSteps;
    return this;
}
```

```
public StepDetails setImportant(boolean important) {
    this.important = important;
    return this;
}
```

```
public StepDetails setStartDate(LocalDateTime startDate) {
    this.startDate = startDate;
}
```

```
        return this;
    }

    public StepDetails setEndDate(LocalDateTime endDate) {
        this.endDate = endDate;
        return this;
    }

    public String getFirstStep() {
        return firstStep;
    }

    public String getResetStudyMechanism() {
        return resetStudyMechanism;
    }

    public String getConsistentStep() {
        return consistentStep;
    }

    public String getSeasonalSteps() {
        return seasonalSteps;
    }

    public String getBasicSteps() {
        return basicSteps;
    }

    public String getMainObjectiveTitle() {
        return mainObjectiveTitle;
    }

    public String getMainGoalTitle() {
        return mainGoalTitle;
    }
}
```

```

public String getMainMaterialTopic() {
    return mainMaterialTopic;
}

public String getMainTask() {
    return mainTask;
}

public Integer getNumberOfSteps() {
    return numberOfSteps;
}

public boolean isImportant() {
    return important;
}

public LocalDateTime getStartDate() {
    return startDate;
}

public LocalDateTime getEndDate() {
    return endDate;
}
}
}

```

Long Parameter List (StudyTaskManager)

Fix the "ExcessiveParameterList" code smell using the "Introduce Parameter Object" refactoring technique.

Lembre, não quero novas classes, só adaptar a que mandei aí

Para resolver o problema de "Excessive Parameter List" no método `setUpWeek`, podemos introduzir um record dentro da classe `StudyTaskManager` para encapsular os parâmetros. Vamos criar um record chamado `WeekSetupAttributes` dentro da própria classe e usá-lo para passar os parâmetros.

Código Gerado:

```
package org.example.studyregistry;

import org.example.studymaterial.Reference;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class StudyTaskManager {
    private static StudyTaskManager instance;
    private StudyMaterial studyMaterial = StudyMaterial.getStudyMaterial();
    List<Registry> registryList;
    List<String> weekResponsibilities = List.of();

    private StudyTaskManager() {
        this.registryList = new ArrayList<Registry>();
    }

    public static StudyTaskManager getStudyTaskManager() {
        if (instance == null) {
            instance = new StudyTaskManager();
        }
        return instance;
    }

    public List<String> getWeekResponsibilities() {
        return weekResponsibilities;
    }

    // Record to encapsulate week setup attributes
    public record WeekSetupAttributes(
        String planName,
        String objectiveTitle,
        String objectiveDescription,
        String materialTopic,
        String materialFormat,
        String goal,
        String reminderTitle,
        String reminderDescription,
        String mainTaskTitle,
```

```

        String mainHabit,
        String mainCardStudy
    ) {}

    public void setUpWeek(WeekSetupAttributes attributes) {
        this.weekResponsibilities = new ArrayList<>();
        this.weekResponsibilities.addAll(Arrays.asList(
            attributes.planName(),
            attributes.objectiveTitle(),
            attributes.objectiveDescription(),
            attributes.materialTopic(),
            attributes.materialFormat(),
            attributes.goal(),
            attributes.reminderTitle(),
            attributes.reminderDescription(),
            attributes.mainTaskTitle(),
            attributes.mainHabit(),
            attributes.mainCardStudy()
        ));
    }

    public void handleSetUpWeek(List<String> stringProperties) {
        WeekSetupAttributes attributes = new WeekSetupAttributes(
            stringProperties.get(0),
            stringProperties.get(1),
            stringProperties.get(2),
            stringProperties.get(3),
            stringProperties.get(4),
            stringProperties.get(5),
            stringProperties.get(6),
            stringProperties.get(7),
            stringProperties.get(8),
            stringProperties.get(9),
            stringProperties.get(10)
        );
        setUpWeek(attributes);
    }

    public void addRegistry(Registry registry) {
        registryList.add(registry);
    }

    public void removeRegistry(Registry registry) {
        registryList.remove(registry);
    }

```

```

    }

    public List<Registry> getRegistryList() {
        return registryList;
    }

    public List<String> searchInRegistries(String text) {
        List<String> response = new ArrayList<>();
        for (Registry registry : registryList) {
            String mix = (registry.getName() != null ? registry.getName() : "");
            if (mix.toLowerCase().contains(text.toLowerCase())) {
                response.add(registry.getName());
            }
        }
        return response;
    }
}

```

```

ele deu problema nisso no getWeekInfo

aqui o código dele:

private void getWeekInfo(){

package org.example.controllers;

import org.example.studymaterial.AudioReference;
import org.example.studymaterial.Reference;
import org.example.studymaterial.TextReference;
import org.example.studymaterial.VideoReference;
import org.example.studyregistry.*;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import static org.example.controllers.MainController.getInput;
import static org.example.controllers.MainController.validateInput;

public class StudyRegistryController {
    StudyTaskManager studyTaskManager =
        StudyTaskManager.getStudyTaskManager();
    StudyMaterial studyMaterial = StudyMaterial.getStudyMaterial();
    private Map<String, Runnable> actions = new HashMap<>();

    public StudyRegistryController() {
        assignActions();
    }
}

```

Vamos corrigir o método `getWeekInfo` para usar o record `WeekSetupAttributes` que encapsula os parâmetros, conforme introduzido na classe `StudyTaskManager`. Isto permitirá que passemos um único objeto com todos os atributos necessários para o método `setUpWeek`.

Código Ajustado:


```

package org.example.studyregistry;

import org.example.studymaterial.Reference;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class StudyTaskManager {
    private static StudyTaskManager instance;
    private StudyMaterial studyMaterial = StudyMaterial.getStudyMaterial();
    List<Registry> registryList;
    List<String> weekResponsibilities = List.of();

    private StudyTaskManager() {
        this.registryList = new ArrayList<Registry>();
    }

    public static StudyTaskManager getStudyTaskManager() {
        if (instance == null) {
            instance = new StudyTaskManager();
        }
        return instance;
    }

    public List<String> getWeekResponsibilities() {
        return weekResponsibilities;
    }

    // Record to encapsulate week setup attributes
    public record WeekSetupAttributes(
        String planName,
        String objectiveTitle,
        String objectiveDescription,
        String materialTopic,
        String materialFormat,
        String goal,
        String reminderTitle,
        String reminderDescription,
        String mainTaskTitle,
        String mainHabit,
        String mainCardStudy
    ) {}

```

```

public void setUpWeek(WeekSetupAttributes attributes) {
    this.weekResponsibilities = new ArrayList<>();
    this.weekResponsibilities.addAll(Arrays.asList(
        attributes.planName(),
        attributes.objectiveTitle(),
        attributes.objectiveDescription(),
        attributes.materialTopic(),
        attributes.materialFormat(),
        attributes.goal(),
        attributes.reminderTitle(),
        attributes.reminderDescription(),
        attributes.mainTaskTitle(),
        attributes.mainHabit(),
        attributes.mainCardStudy()
    ));
}

```

```

public void handleSetUpWeek(List<String> stringProperties) {
    WeekSetupAttributes attributes = new WeekSetupAttributes(
        stringProperties.get(0),
        stringProperties.get(1),
        stringProperties.get(2),
        stringProperties.get(3),
        stringProperties.get(4),
        stringProperties.get(5),
        stringProperties.get(6),
        stringProperties.get(7),
        stringProperties.get(8),
        stringProperties.get(9),
        stringProperties.get(10)
    );
    setUpWeek(attributes);
}

```

```

public void addRegistry(Registry registry) {
    registryList.add(registry);
}

```

```

public void removeRegistry(Registry registry) {
    registryList.remove(registry);
}

```

```

public List<Registry> getRegistryList() {

```

```
    return registryList;
}

public List<String> searchInRegistries(String text) {
    List<String> response = new ArrayList<>();
    for (Registry registry : registryList) {
        String mix = (registry.getName() != null ? registry.getName() : "");
        if (mix.toLowerCase().contains(text.toLowerCase())) {
            response.add(registry.getName());
        }
    }
    return response;
}
}
```